

Credit EurpeBank Answer

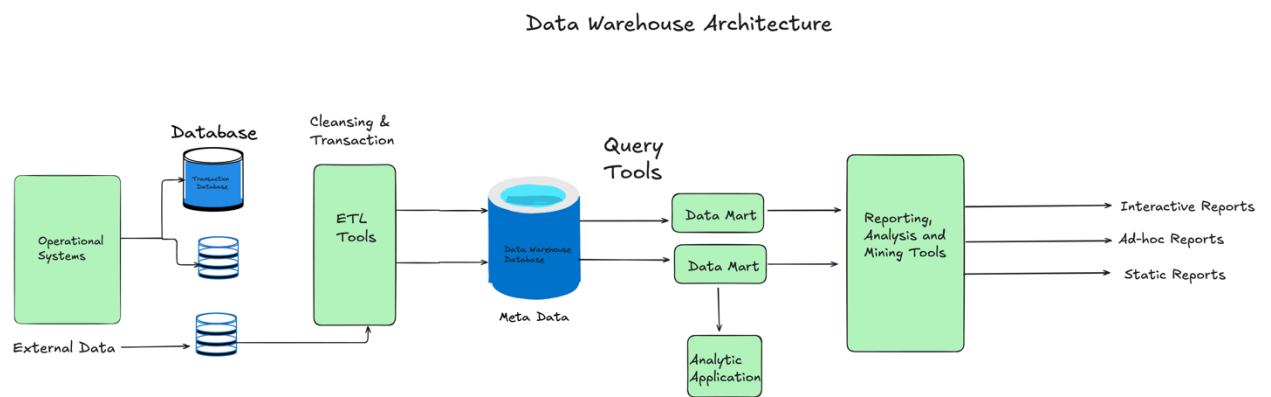
Tolga Boroglu

tolgaboroglu@gmail.com

Part 1 – General Knowledge

1. What is the basic architecture of a typical data warehouse?

Answer:



By Tolga Boroglu

The architecture of a data warehouse consists of various layers to facilitate optimal storage, processing, and analysis of data.

a. Data Source

- **Operational Database:** Include transaccational systems like ERP and CRM this generate data.
- **External Data Sources:** Data can come from external source for example, third-party APIs, social media, and other publicity datasets

b. ETL(Extract, Transform,Load) Process:

- **Extraction:** Data is extracted from various source systems
- **Transform:** The extracted data is cleansed, transformed and formetted
- **Loading:** The tranformed data is loaded into the data warehouse

c. Staging Area:

- It is a temporary storage area where data is held before being transformed and loaded into the warehouse. This method focuses on data cleansing and transformation without affecting the source systems.
- d. Data Warehouse Storage:**
 - **Data Warehouse Database:** It stores, integrates and organizes historical data from various sources for analysis and reporting.
 - **Data Marts:** It is a subset of data warehouse. It provides targeted data access to specific business areas or departments.
- e. Metadata Layer:**
 - Metadata is data about the data stored in the data warehouse. It contains information about the source of the data, transformation processes, structure and usage.
- f. Access and Analysis:**
 - **OLAP(Online Analytical Process) Tools:** It provides users with the opportunity to perform multi-dimensional analysis of business data.
 - **BI(Business Intelligence) Tools:** Provide reporting, querying and data visualization for decision makers.
 - **Data Mining Tools:** Used to discover patterns and insights from large datasets
- g. Presentation Layer:**
 - Interactive reports
 - Ad-hoc analysis
 - Static reports
- h. Security and Management**
 - Ensures data integrity, privacy and compliance with policies and regulations.
 - Users access controls, auditing and monitoring

2. How many type of SCD dimension exist? How do you decide which to choose?

Answer:

SCD is used in the data warehouse to manage and track changes that occur in the data warehouse. There are 6 types of SCD, these are:

1 SCD Type 0 — Dimension is never updated

2 SCD Type 1 — Dimension is overwritten

3 SCD Type 2 — Maintain all the old records for the dimension by versioning the row

4 SCD Type 3 — Maintain an old attribute but in the same row (by adding a previous value column)

5 SCD Type 4 — Maintain current record and older record in two different tables

6 SCD Type 6 — Type 6 is a hybrid of 1,2,3 i.e. $1+2+3 = 6$

How to Decide Which SCD Type to Choose ?

Answer:

- **Business Requirements:** It is evaluated whether the company needs changes or need updated data.
- **Data Volume:** For Types 2 and Types 4, which involve adding more rows, the impact of data volume on performance and storage is evaluated.
- **Data Change Frequency:** Analyze how often changes occur and the impact on data management.
- **Complexity and Maintenance:** Consider the complexity of implementation and maintenance, particularly for hybrid approaches like Type 6.
- **Query Performance:** how different types will affect query performance and using analytical queries.
- **Regulatory Requirements:** Storing data and ensuring compliance with legal requirements.

3. How do you increase SQL Performance of DW on Oracle? What is the effect of adding too many indexes?

- a. Optimize queries
- b. Indexing
- c. Partitioning
- d. Materialized views
- e. Database Configuration
- f. Query caching
- g. Optimize execution plans

Effects of Adding Too Many Indexes

- **Increased Maintenance:** more indexes mean higher maintenance and slower data load times
- **Slower Write Operations:** Inserts, updates, and deletes become slower
- **Increased Storage Needs:** Indexes consume additional disk space
- **Complexity for Optimizer:** Too many indexes can lead to suboptimal
- **Diminishing Returns:** Improve performance and can even degrade it

4. How do you decide to create dimension or fact table?

- **Data type:** Use dimensions descriptive, categorical data; use facts for numeric, quantitative data.
- **Relationships:** Fact tables link to dimension tables; dimensions provide context to facts.
- **Growth and Size:** Fact tables grow with transactions; dimensions change more slowly
- **Analysis Needs:** Determine if data is used for measurement (fact) or context (dimension)

5. How do you increase ETL Performance?

Extraction

- Optimize source queries
- Incremental extraction
- Parallel Processing

Transformation

- Pushdown Processing
- Batch Processing
- Efficient Transformations
- Cache Results

Loading

- Bulk Load
- Index Management
- Partitioning
- Parallel Loaders

6. What is vertical and horizontal scaling? What is CAP theorem?

What is vertical and horizontal scaling?

Vertical scaling increases a single server's capacity by adding resources like CPU and RAM, making it simple but limited by hardware constraints and creating a single point of failure. Horizontal scaling adds more servers to distribute load, improving scalability and fault tolerance but requiring more complex application logic.

What is CAP theorem ?

- The CAP theorem states that a distributed system can only guarantee two of three properties: Consistency (all nodes see the same data), Availability (every request gets a response), and Partition Tolerance (system works despite network issues). Systems must choose between CP (Consistency and Partition Tolerance), AP (Availability and Partition Tolerance), or CA (Consistency and Availability), balancing these trade-offs.

7. What is Conceptual/Logical/Physical design?

- The design process for a database or information system involves three stages: conceptual, logical, and physical design. **Conceptual design** defines the overall structure by identifying key entities and relationships, often visualized using an Entity-Relationship Diagram (ERD). It provides a high-level, abstract view without technical details. **Logical design** translates this into a logical model, organizing data into normalized tables that reflect business

requirements, focusing on reducing redundancy and ensuring data integrity. Finally, **physical design** implements the logical model in a specific database management system (DBMS), detailing tables, indexes, and storage parameters, with a focus on performance and practical deployment. This comprehensive approach ensures the system meets business needs and operates efficiently.

8. Differences between Kimball and Inmon methodologies in short?

- The Kimball and Inmon data warehouse methodologies differ primarily in their architectural approaches. Kimball's methodology follows a bottom-up approach, focusing on developing data warehouses for individual business processes or departments that are then integrated into a unified data warehouse. This method emphasizes dimensional modeling to make the system user-friendly and accessible, providing faster implementation and immediate benefits for specific business areas. It is especially suitable for organizations looking for quick results and solutions tailored to departmental needs. In contrast, the Inmon methodology takes a top-down approach, starting with creating a centralized, enterprise-wide data warehouse that ensures data integration and consistency. This approach uses a normalized relational model to support comprehensive data analysis, with data warehouses derived from the central warehouse to meet specific needs. Inmon's method offers a more integrated and scalable solution, making it ideal for organizations that prioritize data consistency and long-term strategic insights. The choice between these methodologies depends on an organization's goals, resources, and existing data infrastructure.

9. What is Operational Data Store (ODS)?

- An Operational Data Store (ODS) is a database designed for real-time or near-real-time reporting and analysis, providing a consolidated view of current operational data from various sources. Unlike data warehouses, which focus on historical data for long-term analysis, an ODS stores detailed, up-to-date information to support daily business processes and operational reporting. It acts as an intermediary between operational systems and the data warehouse, helping organizations make timely decisions based on the most current data.

10. What can be the types of data sources in data warehouse?

- a. **Transactional Systems:** databases that day to day business transactions
- b. **Operational Databases:** database used for routine business operations and data management (ERP and CRM)
- c. **External Data Source:** market search data, social media , weather data
- d. **Flat Files:** csv, excel or text
- e. **API and Web Services:** application programming or web services that provide real – time or periodic data

- f. **Legacy Systems:** Older systems that may still be in use and require data integration into the modern data warehouse.
- g. **Data Lakes:** large repositories of raw data in various formats that can be processed and integrated into the data warehouse

11. Difference between a data warehouse and a data lake in short?

Data Warehouse:

- **Structured Data:** Stores data in a structured format, usually organized into tables and schemas.
- **Purpose:** Optimized for complex queries, reporting, and analytics on clean, processed data.
- **Schema-on-Write:** Data is structured before it is stored.
- **Usage:** Ideal for business intelligence and historical analysis.

Data Lake:

- **Unstructured and Structured Data:** Stores data in its raw, unprocessed form, including structured, semi-structured, and unstructured data.
- **Purpose:** Provides a scalable repository for storing large volumes of diverse data for future processing and analysis.
- **Schema-on-Read:** Data is structured when it is read or processed.
- **Usage:** Ideal for data exploration, big data analytics, and machine learning.

12. One customer goes to the bank branch and deposits €100 into own account. What amount is registered to branch cash account?

When a customer deposits €100 into his/her account at the bank branch, the following accounting entries are made:

1. **Deposit Account:** The customer's account is increased by €100.
2. **Branch Cash Account:** The branch's cash account is decreased by €100 because the branch is effectively transferring cash from its own reserves to the customer's account.

Thus, the amount registered to the branch's cash account would be a decrease of €100.

Part 2 - SQL Questions

1. Answer

```
SELECT DISTINCT NUM
FROM (
    SELECT
        NUM,
        COUNT(*) OVER (PARTITION BY grp) AS consecutive_count
    FROM (
        SELECT
            NUM,
            ID,
            ROW_NUMBER() OVER (ORDER BY ID) - ROW_NUMBER() OVER (PARTITION BY NUM
            ORDER BY ID) AS grp
        FROM
            Logs
    ) AS grp_table
) AS ConsecutiveNums
WHERE consecutive_count >= 3;
```

2. Answer

```
SELECT
    p.Product_ID,
    COALESCE(MAX(p.Price) KEEP (DENSE_RANK LAST ORDER BY p.As_of_Date), 10) AS
Price
FROM
    (
        SELECT DISTINCT Product_ID FROM Products
        UNION
        SELECT DISTINCT Product_ID FROM (
            SELECT 1 AS Product_ID FROM dual
            UNION ALL
            SELECT 2 AS Product_ID FROM dual
            UNION ALL
            SELECT 3 AS Product_ID FROM dual
        )
    ) p
LEFT JOIN
    Products prod
ON
    p.Product_ID = prod.Product_ID
    AND prod.As_of_Date <= TO_DATE('2024-07-16', 'YYYY-MM-DD')
GROUP BY
    p.Product_ID;
```

3. Answer

SELECT

'Low Salary' AS Category,

COUNT(*) AS Count

FROM

Accounts

WHERE

Income < 20000

UNION ALL

SELECT

'Average Salary' AS Category,

COUNT(*) AS Count

FROM

Accounts

WHERE

Income BETWEEN 20000 AND 50000

UNION ALL

SELECT

'High Salary' AS Category,

COUNT(*) AS Count

FROM

Accounts

WHERE

Income > 50000;

4. Answer

```
SELECT
    d.Name AS Department,
    e.Name AS Employee,
    e.Salary
FROM
    (
        SELECT
            e.ID,
            e.Name,
            e.Salary,
            e.Dept_ID,
            DENSE_RANK() OVER (PARTITION BY e.Dept_ID ORDER BY e.Salary
DESC) AS SalaryRank
        FROM
            Employee e
    ) e
JOIN
    Department d ON e.Dept_ID = d.ID
WHERE
    e.SalaryRank <= 3
ORDER BY
    Department, e.Salary DESC, e.Name;
```

5. Answer

```
SELECT
    TO_CHAR(Trx_date, 'YYYY-MM') AS month,
    Country,
    COUNT(*) AS trans_count,
    COUNT(CASE WHEN Status = 'approved' THEN 1 END) AS approved_count,
    SUM(Amount) AS trans_total_amount,
    SUM(CASE WHEN Status = 'approved' THEN Amount END) AS
approved_total_amount
FROM
    Transaction
GROUP BY
    TO_CHAR(Trx_date, 'YYYY-MM'),
    Country
ORDER BY
    month,
    Country;
```

Part 3 - Design Question:

*Answer what you understand.

You can generate your test data

Design of the Process for Weekly Cost Reporting

Step 1: Data Collection

1. Production Data:

- Source Table: `PRODUCTION\_DATA`
- Details: Contains the quantity produced and the cube (space occupied) for each product. This table is updated monthly.

Columns

- `Product`: Product identifier (`P1`, `P2`, `P3`)
- `Quantity`: Quantity produced (`QP1`, `QP2`, `QP3`)
- `Cube`: Cube space occupied (`CubeP1`, `CubeP2`, `CubeP3`)

2. Administrative Cost Data:

- Source Table: `ADMIN\_COST`
- Details: Contains administrative costs, which are allocated to products based on their cube.

Columns:

- `Cost\_type`: Type of cost (e.g., `Administrative expenses`)
- `Amount`: Monthly cost (`CostAdm`)

3. Production Cost Data:

- Cost per unit data: These are constants or can be fetched from a separate table.
- `CostP1/unit`, `CostP2/unit`, `CostP3/unit`: Production cost per unit for each product.

Step 2: Data Transformation

1. Calculate Weekly Production Costs:

- Multiply the quantity produced by the cost per unit for each product.
- Example: $\text{Weekly_Cost_P1} = (\text{QP1} / 4) * \text{CostP1/unit}$ (assuming equal production across 4 weeks).

2. Allocate Administrative Costs:

- Allocate the monthly administrative cost to products based on the proportion of cube space occupied by each product.
- Example: $\text{Allocated_Adm_Cost_P1} = (\text{CubeP1} / \text{Total_Cube}) * (\text{CostAdm} / 4)$ (for weekly allocation).

Step 3: Data Loading

1. Fact Table:

- Table Name: `FACT_COST`

Columns:

- `Product_ID`: `P1`, `P2`, `P3`
- `Week`: Week number
- `Production_Cost`: Weekly production cost for the product
- `Admin_Cost`: Allocated weekly administrative cost for the product
- `Total_Cost`: Sum of production and administrative costs

2. Dimension Tables:

Product Dimension:

- `Product\_ID` : `P1`, `P2`, `P3`
- `Product\_Name` : Name of the product

- Time Dimension:

- `Date` : Date of the week
- `Week\_Number` : Week number of the year
- `Month` : Month

Step 4: Reporting

1. Reporting Layer:

- A reporting tool like Power BI, Tableau, or Excel can be used to visualize the data.
- Reports include total weekly costs and individual product costs.

SQL Queries

1) SQL Query to Get Total Weekly Cost (Administrative + Production Cost) for Company A

```
```sql
```

```
SELECT
```

```
 Week,
```

```
 SUM(Production_Cost + Admin_Cost) AS Total_Weekly_Cost
```

```
FROM
```

```
 FACT_COST
```

```
GROUP BY
```

```
 Week
```

```
ORDER BY
```

Week;

2) SQL Query to Get Total Weekly Cost (Administrative + Production Cost) for Each Product

```
```sql
```

```
SELECT
```

```
    Product_ID,
```

```
    Week,
```

```
    (Production_Cost + Admin_Cost) AS Product_Weekly_Cost
```

```
FROM
```

```
    FACT_COST
```

```
ORDER BY
```

```
    Product_ID, Week;
```

```
```
```

Test Data (Example)

PRODUCTION\_DATA

| Product | Quantity | Cube |
|---------|----------|------|
|---------|----------|------|

| Product | Quantity | Cube |
|---------|----------|------|
|---------|----------|------|

|    |     |     |
|----|-----|-----|
| P1 | 400 | 200 |
|----|-----|-----|

|    |     |     |
|----|-----|-----|
| P2 | 300 | 150 |
|----|-----|-----|

|    |     |     |
|----|-----|-----|
| P3 | 500 | 250 |
|----|-----|-----|

## ADMIN\_COST

| Cost_type            | Amount |
|----------------------|--------|
| Administrative costs | 10000  |

Sample Output:

Assume:

- `CostP1/unit = 10`, `CostP2/unit = 15`, `CostP3/unit = 12`
- Weekly costs calculated by dividing the monthly data by 4.

## FACT\_COST

| Product_ID | Week | Production_Cost | Admin_Cost | Total_Cost |
|------------|------|-----------------|------------|------------|
| P1         | 1    | 1000            | 500        | 1500       |
| P2         | 1    | 1125            | 375        | 1500       |
| P3         | 1    | 1500            | 625        | 2125       |
| P1         | 2    | 1000            | 500        | 1500       |
| P2         | 2    | 1125            | 375        | 1500       |
| P3         | 2    | 1500            | 625        | 2125       |

This structured approach ensures accurate and timely weekly cost reporting, supporting profitability analysis for Company A.